МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №2
з дисципліни "Основи Програмної Інженерії"

Виконав
ст. гр. ПЗПІ-25-3
Коцарєв.М.О

Прийняв:
В.Гребенюк

Харків 2026

ЛАБОРАТОРНА РОБОТА № 2

МОДЕЛЮВАННЯ ПРОГРАМНОЇ СИСТЕМИ ЗАСОБАМИ UML (UNIFIED MODELING LANGUAGE)

2.1 Мета роботи

Набуття практичних навичок моделювання програмних систем із використанням уніфікованої мови моделювання UML (Unified Modeling Language). Оволодіння методами побудови діаграм прецедентів (Use Case Diagram), діаграм класів (Class Diagram) та діаграм послідовності (Sequence Diagram). Формування вміння забезпечувати трасовність (traceability) від функціональних вимог до моделей проєктування.

У результаті виконання роботи здобувач має продемонструвати здатність: обирати предметну область та формулювати функціональні вимоги; застосовувати нотацію UML 2.5 для візуального моделювання архітектури програмної системи; будувати матрицю трасовності, що пов’язує вимоги, прецеденти, класи та сценарії взаємодії.

2.2 Хід роботи

2.2.1 Предметна область

DeltaPhys - платформа для пошуку партнерів та професійного нетворкінгу в екстремальних видах спорту.

2.2.2 Функціональні вимоги

FR-01: Система повинна дозволяти реєстрацію та автентифікацію користувачів через email.

FR-02: Система повинна надавати можливість створення анкет («сесій») для пошуку партнерів.

FR-03: Система повинна відображати публічну карту доступних спортивних подій.

FR-04: Система повинна дозволяти верифікацію профілю атлета.

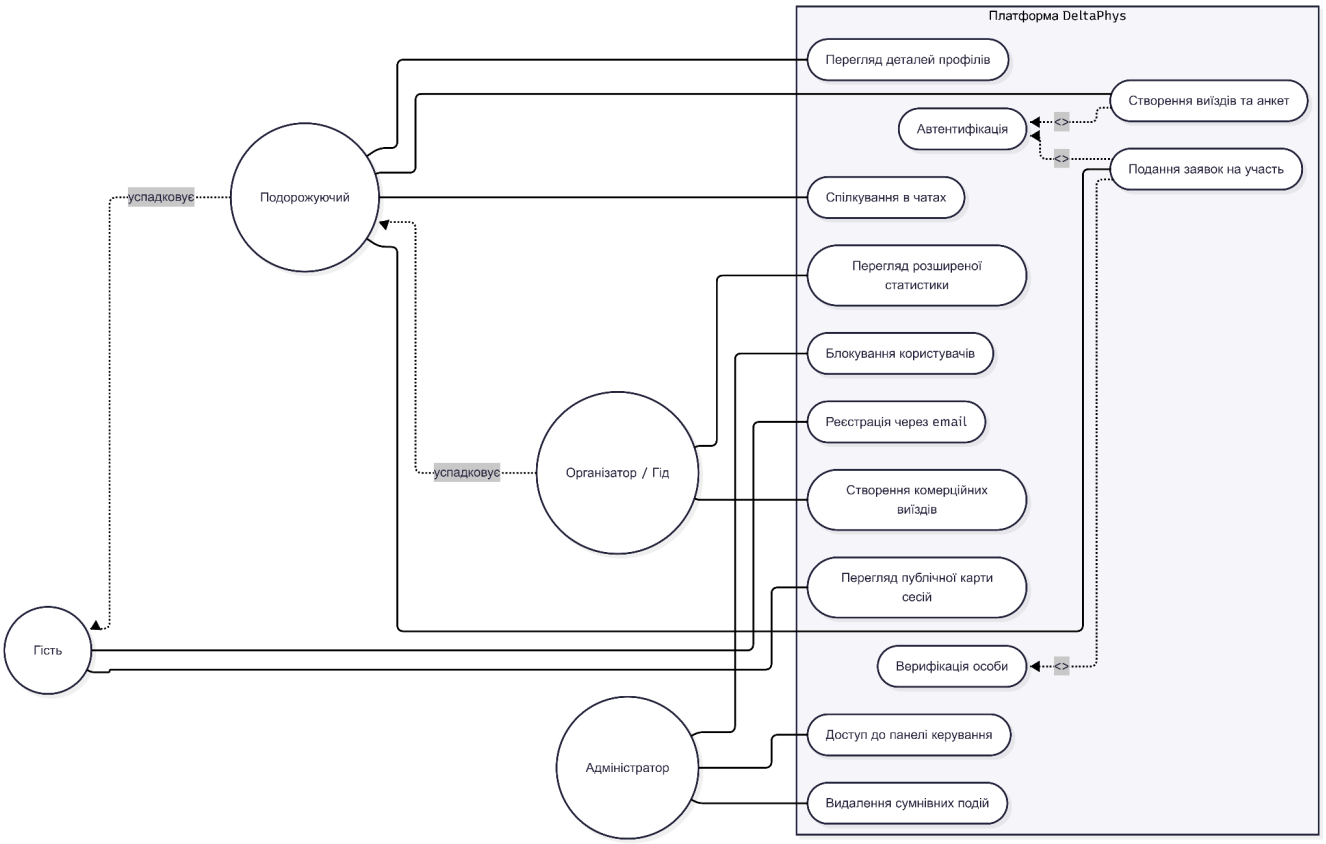
FR-05: Система повинна забезпечувати функціонал чатів для комунікації між учасниками сесії.

2.2.3 Діаграма прецедентів

Код

```
---
config:
  layout: elk
---
flowchart RL
 subgraph DeltaPhys["Платформа DeltaPhys"]
    direction TB
        UC1(["Перегляд публічної карти сесій"])
        UC2(["Реєстрація через email"])
        UC13(["Автентифікація"])
        UC3(["Перегляд деталей профілів"])
        UC4(["Спілкування в чатах"])
        UC5(["Створення виїздів та анкет"])
        UC6(["Подання заявок на участь"])
        UC7(["Верифікація особи"])
        UC8(["Створення комерційних виїздів"])
        UC9(["Перегляд розширеної статистики"])
        UC10(["Доступ до панелі керування"])
        UC11(["Блокування користувачів"])
        UC12(["Видалення сумнівних подій"])
  end
    UC1 --- G(("Гість"))
    UC2 --- G
    UC3 --- T(("Подорожуючий"))
    UC4 --- T
    UC5 --- T
    UC6 --- T
    UC8 --- O(("Організатор / Гід"))
    UC9 --- O
    UC10 --- Admin(("Адміністратор"))
    UC11 --- Admin
    UC12 --- Admin
    O -. успадковує .-> T
    T -. успадковує .-> G
    UC5 -. &lt;&gt; .-> UC13
    UC6 -. &lt;&gt; .-> UC13 & UC
```

Діаграма



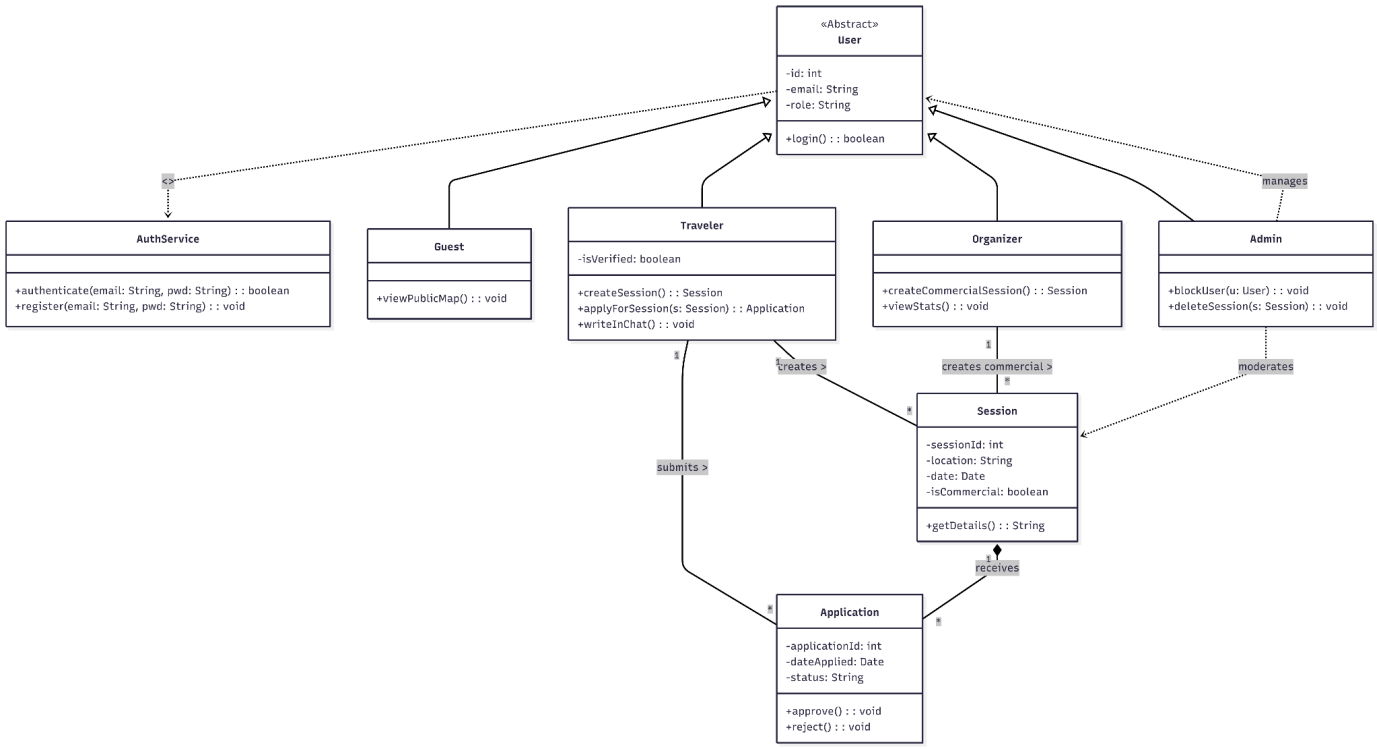
2.2.4 Діаграма класів

Код

```
---
config:
  layout: dagre
---
classDiagram
    class AuthService {
        +authenticate(email: String, pwd: String): boolean
        +register(email: String, pwd: String): void
    }

    class User {
        <<Abstract>>
        -id: int
        -email: String
        -role: String
        +login(): boolean
    }

    class Guest {
        +viewPublicMap(): void
    }

    class Traveler {
        -isVerified: boolean
        +createSession(): Session
        +applyForSession(s: Session): Application
        +writeInChat(): void
```

```
}

class Organizer {
    +createCommercialSession(): Session
    +viewStats(): void
}

class Admin {
    +blockUser(u: User): void
    +deleteSession(s: Session): void
}

class Session {
    -sessionId: int
    -location: String
    -date: Date
    -isCommercial: boolean
    +getDetails(): String
}

class Application {
    -applicationId: int
    -dateApplied: Date
    -status: String
    +approve(): void
    +reject(): void
}
User <|-- Guest
User <|-- Traveler
User <|-- Organizer
User <|-- Admin
User ..> AuthService : <<uses>>
Traveler "1" -- "*" Session : creates >
Traveler "1" -- "*" Application : submits >
Organizer "1" -- "*" Session : creates commercial >
Session "1" *-- "*" Application : receives
Admin ..> User : manages
Admin ..> Session : moderates
```

Діаграма



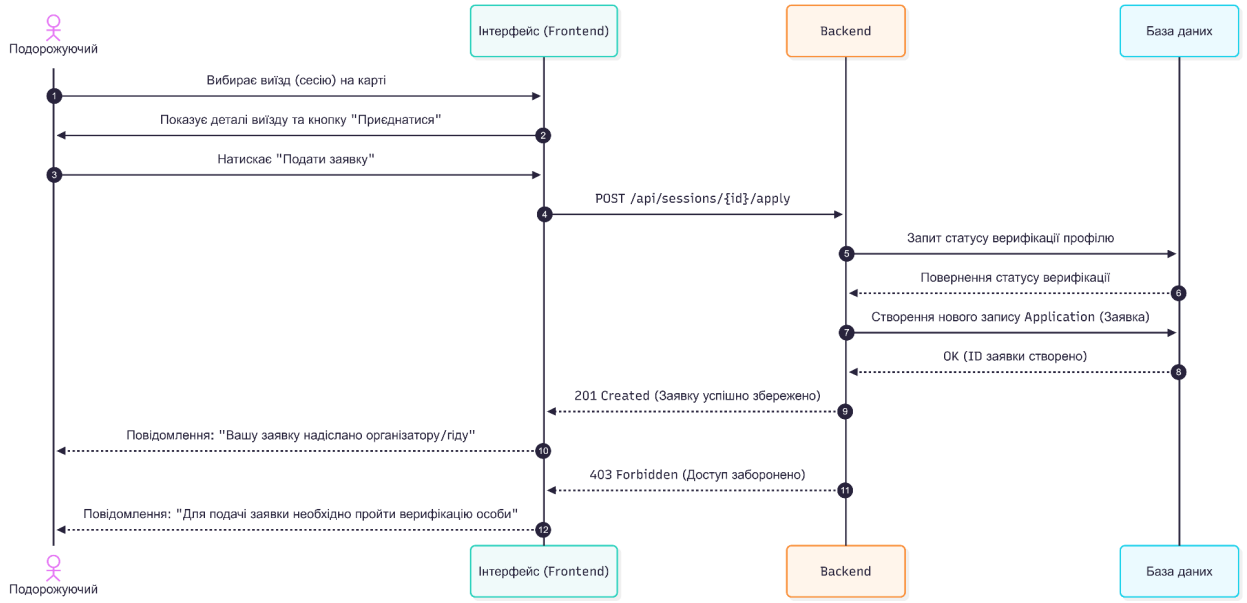
2.2.5Діаграма послідовності

Код

```
---
config:
  theme: redux-color
  look: neo
---
sequenceDiagram
    actor Traveler as Подорожуючий
    participant UI as Інтерфейс (Frontend)
    participant System as Backend
    participant DB as База даних
    autonumber

    Traveler->>UI: Вибирає виїзд (сесію) на карті
    UI->>Traveler: Показує деталі виїзду та кнопку "Приєднатися"
    Traveler->>UI: Натискає "Подати заявку"
    UI->>System: POST /api/sessions/{id}/apply
    System->>DB: Запит статусу верифікації профілю
    DB-->>System: Повернення статусу верифікації
```

```
    %% Комбінований фрагмент умовного розгалуження (alt) згідно з методичкою
    System->>DB: Створення нового запису Application (Заявка)
    DB-->>System: OK (ID заявки створено)
    System-->>UI: 201 Created (Заявку успішно збережено)
    UI-->>Traveler: Повідомлення: "Вашу заявку надіслано організатору/гіду"
    System-->>UI: 403 Forbidden (Доступ заборонено)
    UI-->>Traveler: Повідомлення: "Для подачі заявки необхідно пройти верифікацію особи"
```

Діаграма



2.2.6 Матриця трасовності

Вимога	Use Case	Класи	Sequence
FR-01: Система повинна дозволяти реєстрацію та автентифікацію користувачів через email.	UC2: Реєстрація через email	AuthService, User	

FR-02: Система повинна надавати можливість створення анкет («сесій») для пошуку партнерів.	UC5: Створення виїздів та анкет UC8: Створення комерційних виїздів	Traveler, Organizer, Session	
FR-03: Відображення публічної карти доступних спортивних подій	UC1: Перегляд публічної карти сесій UC6: Подання заявок на участь	Guest, Session, Application	Вибирає виїздна карті, показує деталі виїзду та кнопку "Приєднатися" Створення нового запису Application
FR-04: Верифікація профілю атлета	UC7: Верифікація особи	Traveler (атрибут isVerified)	Перевірка статусу верифікації профілю. Профіль верифіковано
FR-05: Функціонал чатів для комунікації між учасниками сесії	UC4: Спілкування в чатах	Traveler (метод writeInChat())	